Axiomatics and Multi-Diagram UML of Bitcoin Primitives, Sub-Protocols, Layers, and a Kapnet-Aware Overlay

Abstract

This document restates the prior layered UML in a denser formal style and expands it into multiple TikZ package diagrams. Bitcoin is modeled as a stratified system whose lower layers define conserved validity and whose higher layers define policy, composition, channels, and application commitments. A Kapnet-aware extension is then added as a subordinate meta-consensus layer containing weakwork claims, TXXM packets, knots, braids, ACT digests, overlay accumulators, provider nodules, and multi-base-chain coordination rules.

The central thesis is simple: Bitcoin remains the base validity and settlement substrate; higher layers introduce increasingly rich coordination surfaces; a Kapnet-style overlay formalizes replayable semantic and cypher-economic structures above the base chain without collapsing the distinction between base consensus and overlay semantics.

Contents

1 Notation and Scope **3**

2 Layer Inventory **3**

3 Axiomatic Core **4**

4 Definitions **4**

5 Formal Decomposition of Bitcoin Layers **5**

5.1 $\mathcal{L}_0$: Mathematical / Cryptographic Foundations 5

5.2 $\mathcal{L}_1$: Consensus-Native Ledger Primitives 6

5.3 $\mathcal{L}_2$: Script / Spending Layer 6

5.4 $\mathcal{L}_3$: Encodings and Declarative Surfaces 6

5.5 $\mathcal{L}_4$: Wallet / Signing / Policy Layer 6

5.6 $\mathcal{L}_5$: Validation / Relay Layer 6

5.7 $\mathcal{L}_6$: P2P Networking Layer 7

5.8 $\mathcal{L}_7$: Mining / Issuance / Time Layer . . . 7
5.9 $\mathcal{L}_8$: Deployment / Chain Evolution Layer . . . 7
5.10 $\mathcal{L}_9$: Canonical Bitcoin Sub-Protocols . . . 7
5.11 $\mathcal{L}_{10}$: Higher Bitcoin Protocols . . . 7
5.12 $\mathcal{L}_{11}$: Overlay / Application Layer . . . 7

6 Kapnet Extension as $\mathcal{L}_{12}$ 7
6.1 Interpretive Claim . . . 7
6.2 Kapnet Object Family . . . 8
6.3 Kapnet Sub-Layers . . . 8

7 Formal Relations Between Layers 8

8 Textual UML Dependency Table 9

9 Multi-Diagram TikZ UML Package Views 11
9.1 Diagram 1: Base Stack . . . 11
9.2 Diagram 2: Consensus-Native Core . . . 12
9.3 Diagram 3: Spend Authorization Surface . . . 12
9.4 Diagram 4: Wallet and Policy Surface . . . 12
9.5 Diagram 5: Validation, Relay, and Mining . . . 13
9.6 Diagram 6: Canonical BIP Package View . . . 13
9.7 Diagram 7: Higher Protocols Above Settlement . . . 13
9.8 Diagram 8: Kapnet Overlay Package View . . . 14
9.9 Diagram 9: Multi-Chain Qualified Overlay Namespace . . . 14

10 Main Structural Theorems 14

11 AggWit and Extended Accounting as an Optional Surface 15

12 Interpretive Summary 15

13 Conclusion 16

1 Notation and Scope

Let

$$\mathcal{L}_0, \mathcal{L}_1, \ldots, \mathcal{L}_{12}$$

denote the ordered protocol layers. Lower indices indicate stronger proximity to consensus-native validity.

Let

$$\mathcal{B}$$

denote the set of Bitcoin-valid blocks under a fixed consensus rule set.

Let

$$\mathcal{T}$$

denote the set of Bitcoin-valid transactions.

Let

$$\mathcal{U}$$

denote the induced UTXO set at a given chainstate.

Let

$$\mathcal{O}$$

denote an overlay state space, interpreted by deterministic replay rules subordinate to Bitcoin-valid anchors.

We will use the phrase *base-valid* to mean valid under Bitcoin consensus, and *overlay-valid* to mean valid under an additional deterministic overlay interpreter whose admissible anchors are drawn from base-valid objects.

2 Layer Inventory

Lyr	**Name**	**Interpretation**
0	Mathematical Foundations	Hashes, signatures, Merkle commitments, PoW comparison, retarget arithmetic, timelock arithmetic.
1	Ledger Primitives	Headers, blocks, transactions, inputs, outputs, UTXOs, mempool, chain tip.
2	Script / Spend Layer	Script VM, output templates, timelocks, SegWit, Taproot, Tapscript.
3	Encoding / Description Layer	Base58Check, Bech32, Bech32m, WIF, descriptors, Miniscript, derivation notation.
4	Wallet / Signing Layer	Seeds, HD trees, PSBT, signers, coin selection, change policy, descriptor wallets.
5	Validation / Relay Layer	Consensus validation, standardness, mempool policy, reorg logic, fee estimation.
6	P2P Networking Layer	Discovery, handshake, relay families, sync strategies, compact blocks, encrypted transport.

7	Mining / Issuance Layer	Block assembly, coinbase, fees, subsidy, halving, target search, pools.
8	Chain Evolution Layer	Forks, activations, versionbits, buried deployments, user-coordination.
9	Canonical Sub-Protocols	BIP-defined standard modules such as P2SH, SegWit, PSBT, Taproot, BIP324.
10	Higher Bitcoin Protocols	Lightning, DLCs, CoinJoin, Fedimint, Ark, statechains, covenant research.
11	Overlay / Application Layer	Timestamping, proof systems, Ordinals, inscriptions, Runes, RGB, anchoring.
12	Kapnet Overlay Layer	Weakwork, TXXM, knots, braids, ACTs, accumulators, provider nodules, multi-tip coordination.

3 Axiomatic Core

Axiom 1 (Hash Commitment Asymmetry)**.** *A succinct digest may bind to a much larger body of data than the digest itself reveals, and efficient verification may depend on partial witnesses rather than full data reconstruction.*

Axiom 2 (Signature Authorization)**.** *A spend authorization relation is defined by a public predicate and a witness whose validity is efficiently checkable and infeasible to forge without the requisite signing capability.*

Axiom 3 (UTXO Conservation with Issuance Exception)**.** *For every non-coinbase transaction, the sum of spent values is at least the sum of created values; for coinbase transactions, the excess is bounded by subsidy plus fees.*

Axiom 4 (Chain Selection Under Work Ordering)**.** *Among competing valid histories, nodes adopt a chain selection rule derived from accumulated proof-of-work and local validity.*

Axiom 5 (Consensus-Policy Separation)**.** *There exist predicates enforced by all validating nodes as consensus and stricter predicates enforced only as relay or mining policy.*

Axiom 6 (Semantic Superordination)**.** *An overlay may define richer state transitions than base consensus provided every accepted overlay transition is grounded in admissible base anchors and is replayable by a deterministic interpreter.*

Axiom 7 (Anchored Optionality)**.** *A commitment surface used by higher protocols need not itself be consensus-semantic beyond its base interpretation; higher meaning may be assigned by subordinate protocols.*

4 Definitions

Definition 1 (Block Header)**.** *A block header is the tuple*

$$H = (v, h_{prev}, r_{merkle}, t, nBits, \nu)$$

consisting of a version, previous block reference, Merkle root, timestamp, compact target field, and nonce.

Definition 2 (Block)**.** *A block is a pair*

$$B = (H, \mathbf{tx})$$

where H is a valid block header and $\mathbf{tx}$ is an ordered transaction list whose Merkle commitment matches r_{merkle}.

Definition 3 (Transaction)**.** *A transaction is a finite state transition object*

$$\tau = (v, \mathbf{vin}, \mathbf{vout}, \ell)$$

with version v, ordered inputs, ordered outputs, and locktime ℓ.

Definition 4 (UTXO Set)**.** *Given a valid chain prefix C, the UTXO set $\mathcal{U}(C)$ is the set of outputs created in C and not yet spent in C.*

Definition 5 (Script Predicate)**.** *A script predicate is a computable acceptance rule on a witness and transaction context that determines whether a spend is authorized.*

Definition 6 (Descriptor)**.** *A descriptor is a declarative description of a spend template and associated key or derivation material sufficient to derive addresses, scripts, or signing policy.*

Definition 7 (Overlay Anchor)**.** *An overlay anchor is a base-valid object or digest reference thereto used as an admissible commitment point for overlay interpretation.*

Definition 8 (TXXM)**.** *A TXXM is a transaction-crossmail overlay packet containing action, routing, validation, and payload fields, replayable under overlay rules and optionally anchored to one or more base-valid events.*

Definition 9 (Knot)**.** *A knot is a structured local aggregate of overlay packets, claims, validations, or references treated as a meaningful checkpointable unit by overlay rules.*

Definition 10 (Braid)**.** *A braid is a partially ordered family of knots or claims whose relation structure is protocol-relevant.*

Definition 11 (ACT)**.** *An Amethyst Control Tip is a deterministic overlay digest associated with a designated progress point in one or more anchored chains.*

Definition 12 (Weakwork Claim)**.** *A weakwork claim is a low-cost overlay competition object whose admissibility and reward are determined by deterministic overlay rules rather than by base consensus.*

5 Formal Decomposition of Bitcoin Layers

5.1 $\mathcal{L}_0$: Mathematical / Cryptographic Foundations

Core objects:

$$\{\text{SHA-256}, \text{Double-SHA-256}, \text{RIPEMD-160}, \text{secp256k1}, \text{ECDSA}, \text{Schnorr}, \text{Merkle}, \text{PoW}, \text{Retarget}, \text{Timelock Arithme}$$

These objects are not directly economic state; they are admissibility machinery.

Lemma 1 (Digest Dependence)**.** *Every Bitcoin-valid header depends on hash primitives and target comparison logic.*

Proof. Header validity requires computing the block hash and comparing it to a target encoded by $nBits$. This is impossible without the hash primitive and target arithmetic. □

5.2 $\mathcal{L}_1$: Consensus-Native Ledger Primitives

Core objects:

$\{\text{BlockHeader}, \text{Block}, \text{Transaction}, \text{TxInput}, \text{TxOutput}, \text{OutPoint}, \text{UTXO}, \text{CoinbaseTx}, \text{Mempool}, \text{ChainTip}\}$.

Proposition 1 (Containment Relation)**.** *A block contains transactions; a transaction contains inputs and outputs; a UTXO is induced by an output together with chainstate non-spend status.*

5.3 $\mathcal{L}_2$: Script / Spending Layer

Core objects:

$\{\text{Script VM}, \text{Opcodes}, \text{P2PK}, \text{P2PKH}, \text{P2SH}, \text{Bare Multisig}, \text{OP_RETURN}, \text{CLTV}, \text{CSV}, \text{SegWit v0}, \text{Taproot}, \text{Taps}$

Theorem 1 (Spend Authorization Factorization)**.** *A successful spend factors into:*

$$\textit{referenced output} + \textit{context} + \textit{witness} + \textit{script rule} \longrightarrow \textit{accept or reject.}$$

Proof. Every input references a prior output. That output provides the locking predicate. The spending transaction and witness provide the unlocking material and evaluation context. The VM deterministically accepts or rejects. □

5.4 $\mathcal{L}_3$: Encodings and Declarative Surfaces

Core objects:

$\{\text{Base58Check}, \text{Bech32}, \text{Bech32m}, \text{WIF}, \text{x-only pubkeys}, \text{xpub/xprv}, \text{BIP32 paths}, \text{Descriptors}, \text{Miniscript}\}$.

These do not generally alter consensus; they encode or describe admissible use of lower-layer objects.

5.5 $\mathcal{L}_4$: Wallet / Signing / Policy Layer

Core objects:

$\{\text{Wallet}, \text{HDWallet}, \text{SeedPhrase}, \text{MasterKey}, \text{Account}, \text{CoinSelection}, \text{ChangeOutput}, \text{PSBT}, \text{Signer}, \text{HardwareWalle}$

Lemma 2 (Policy Non-Identity)**.** *Distinct wallets may implement different coin selection and change strategies while still constructing base-valid transactions.*

5.6 $\mathcal{L}_5$: Validation / Relay Layer

Core objects:

$\{\text{ConsensusRules}, \text{Standardness}, \text{TxValidation}, \text{BlockValidation}, \text{UTXOSet}, \text{Chainstate}, \text{Reorg}, \text{FeeEstimation}, \text{Mem}$

Theorem 2 (Consensus / Policy Strict Inclusion)**.** *There exist transactions that are consensus-valid but non-standard under common relay policy.*

Proof. The existence of non-standard but valid constructions follows from the distinction between what must be accepted in blocks and what nodes choose to relay or mine by local policy. □

5.7 $\mathcal{L}_6$: P2P Networking Layer

Core objects:

{Peer, Handshake, PeerDiscovery, DNSSeeds, addr, inv, getdata, tx relay, block relay, header relay, compact blocks,

This layer carries data; it does not itself define validity.

5.8 $\mathcal{L}_7$: Mining / Issuance / Time Layer

Core objects:

{Miner, MiningTemplate, BlockAssembler, CoinbaseCommitments, Subsidy, Fees, Halving, DifficultyEpoch, Target,

Proposition 2 (Coinbase Speciality)**.** *The coinbase transaction is the unique transaction class permitted to create new base units under subsidy rules.*

5.9 $\mathcal{L}_8$: Deployment / Chain Evolution Layer

Core objects:

{Genesis, BestChain, Fork, SoftFork, HardFork, Activation, VersionBits, BuriedDeployment, UASF-like coordinatio

5.10 $\mathcal{L}_9$: Canonical Bitcoin Sub-Protocols

Representative BIP-linked modules:

{BIP16, BIP32, BIP39, BIP65, BIP68, BIP112, BIP141, BIP143, BIP157/158, BIP173, BIP174, BIP340, BIP341, BI

5.11 $\mathcal{L}_{10}$: Higher Bitcoin Protocols

Representative protocols:

{Lightning, DLC, CoinJoin, PayJoin, Fedimint, Ark, Statechain, Sidechain Concepts, Covenant Research}.

5.12 $\mathcal{L}_{11}$: Overlay / Application Layer

Representative systems:

{Timestamping, Proof-of-Reserves, Colored Coin Style Systems, Ordinals, Inscriptions, Runes, RGB, Nostr Anchori

6 Kapnet Extension as $\mathcal{L}_{12}$

6.1 Interpretive Claim

Axiom 8 (Kapnet Subordination)**.** *Every Kapnet-valid state transition must be grounded in admissible anchors that are themselves base-valid or derived from base-valid objects through deterministic indexing.*

Axiom 9 (Bounded Replay)**.** *Every accepted overlay transition must be replayable by a bounded deterministic interpreter given the overlay transcript and anchor set required by the protocol.*

Axiom 10 (Checkpoint Compression)**.** *Overlay history may be compressed into succinct checkpoint digests without losing the ability to verify admitted state transitions against the committed transcript and witnesses.*

6.2 Kapnet Object Family

Let

$$\mathcal{K} = \{\text{WeakworkClaim}, \text{TXXM}, \text{Knot}, \text{Braid}, \text{Accumulator}, \text{ACT}, \text{ProviderNodule}, \text{Guild}, \text{Room}, \text{Submission}\}.$$

Definition 13 (Overlay Accumulator)**.** *An overlay accumulator is a digestible summary of a finite set of overlay events, claims, or state transitions sufficient to support checkpointing and witness-based verification.*

Definition 14 (Provider Nodule)**.** *A provider nodule is an overlay actor or service role that relays, prices, indexes, validates, or markets overlay functions such as weakwork submission, routing, or checkpoint servicing.*

6.3 Kapnet Sub-Layers

For clarity, decompose $\mathcal{L}_{12}$ into:

$$\mathcal{L}_{12A}, \ldots, \mathcal{L}_{12F}$$

as follows.

Sub-layer	**Role**
$12A$	Overlay message forms: TXXM packets, validation packets, governance packets, references, routing instructions.
$12B$	Overlay execution: KScript-like interpretation, bounded resource accounting, dependency checks, domain constraints.
$12C$	Overlay commitment: knots, braids, accumulators, ACT digests, checkpoint roots.
$12D$	Overlay economics: burn, bond, postage, stake, change, weakwork pricing, deterministic lotteries.
$12E$	Overlay networking: provider nodules, relay meshes, persistence, synchronization, checkpoint dissemination.
$12F$	Multi-base-chain coordination: chain-qualified namespaces, auxiliary anchoring, merge-mined overlays, synchronized ACT lineages.

7 Formal Relations Between Layers

Theorem 3 (Layer Dependence)**.** *For each $i \in \{1, \ldots, 12\}$, layer $\mathcal{L}_i$ depends on structures furnished by one or more lower layers $\mathcal{L}_j$ with $j < i$.*

Proof. By inspection:

- Ledger objects require cryptographic primitives.
- Script spends require ledger references and cryptographic witnesses.
- Encodings describe keys, scripts, or outputs already furnished below them.
- Wallet policy constructs transactions and witnesses for lower-layer validity.
- Validation checks lower-layer admissibility.
- Networking transports lower-layer and peer-coordination data.
- Mining assembles lower-layer objects into candidate blocks.
- Deployment alters or constrains lower-layer validity regimes.
- BIP sub-protocols refine or standardize behaviors atop earlier strata.
- Higher protocols consume settlement and contract surfaces from below.
- Overlay applications depend on base commitments and ordering.
- Kapnet depends on admissible anchors and deterministic interpretation above them.

□

Corollary 1 (No Free Semantic Lift)**.** *No higher-layer semantic claim can force base validity unless adopted into the base consensus or policy layers.*

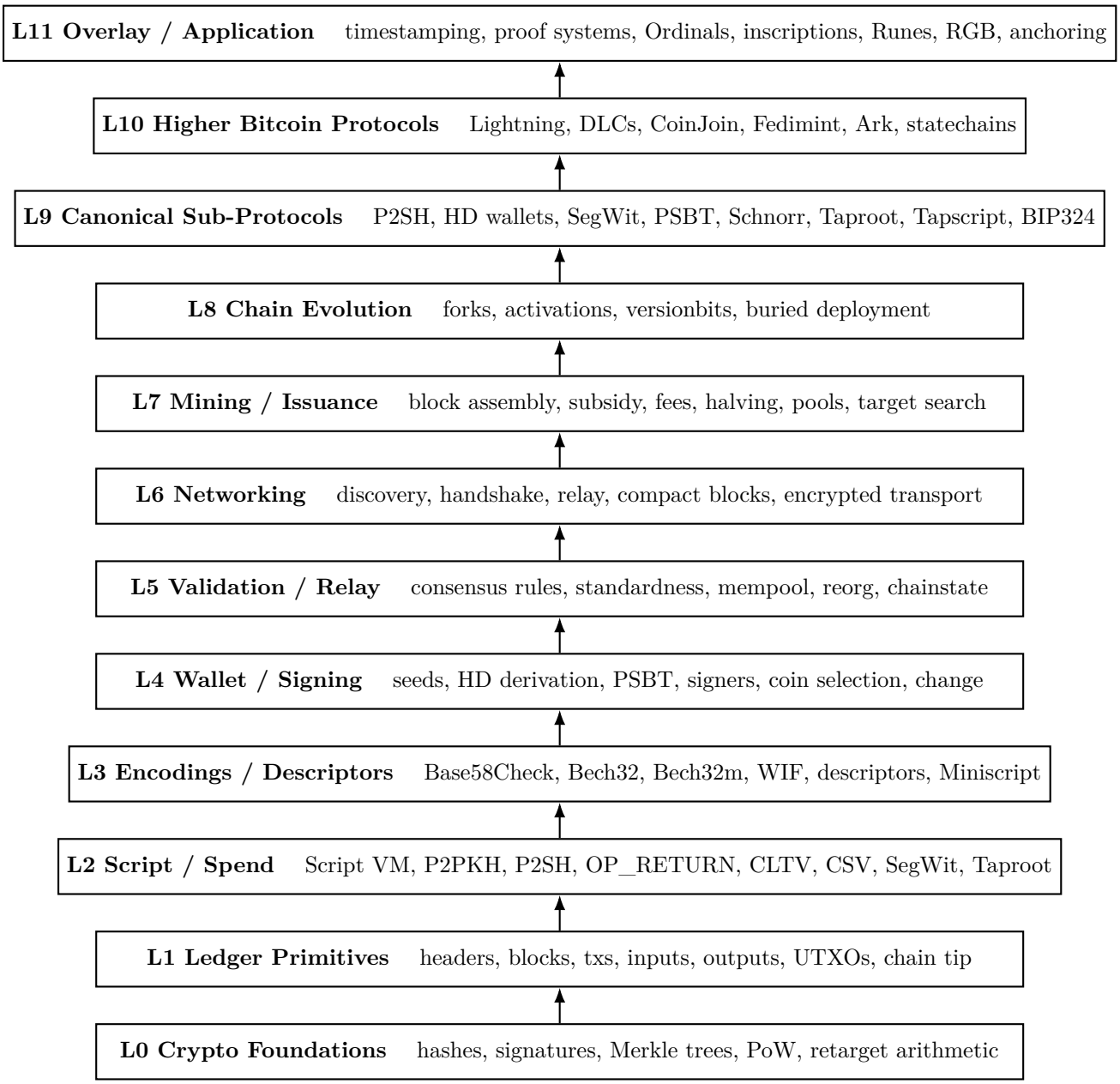
8 Textual UML Dependency Table

Object	**Depends on / Contains / Commits to**
Double-SHA256	SHA-256
Merkle Tree	Hash primitive family
PoW	Double-SHA256, target comparison
BlockHeader	Previous block reference, Merkle root, target field, nonce
Block	BlockHeader, ordered transactions
Transaction	Inputs, outputs, version, locktime
TxInput	OutPoint, unlocking material, sequence, optional witness
TxOutput	Value, locking predicate
UTXO	TxOutput with unspent chainstate status
P2PKH / P2SH / Taproot Output	ScriptPubKey template family
SegWit	Witness separation and digest redesign
Taproot	x-only keys, Schnorr, script tree commitments
Descriptor	Spend template, key origin data, derivation logic
PSBT	Unsigned transaction plus coordination metadata

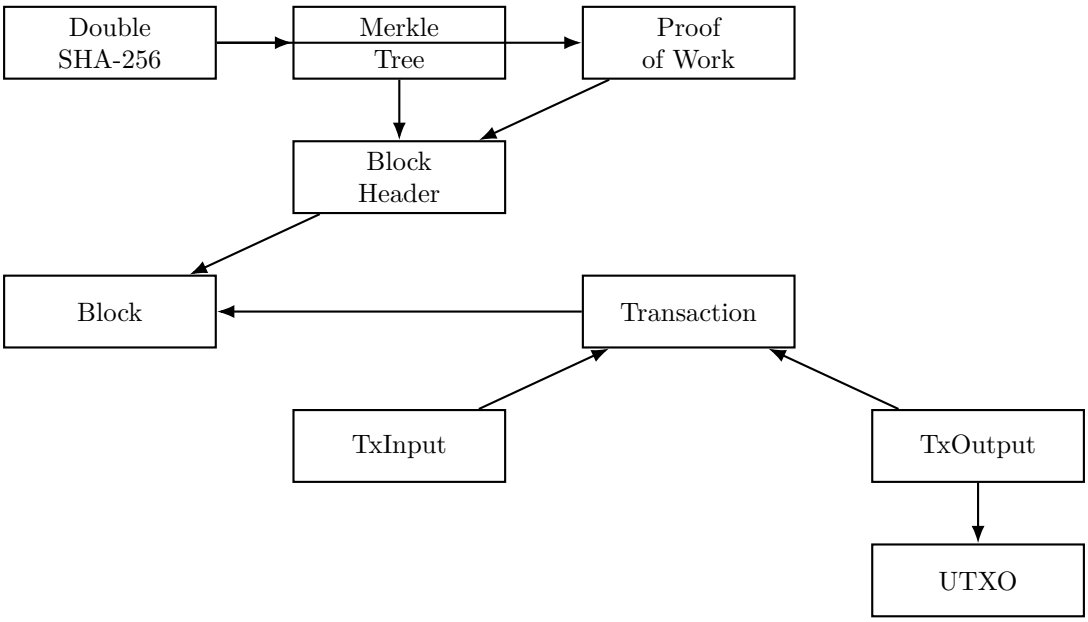
Wallet	Key material, descriptors, coin selection, policy
Full Node	Consensus rules, tx validation, block validation, chainstate
Mempool	Consensus-valid unconfirmed txs filtered by policy
Peer	Transport, relay messages, negotiated capabilities
Mining Template	Candidate tx set, coinbase data, policy ordering
Coinbase Surface	Subsidy, fee capture, optional auxiliary commitments
Lightning Channel	Funding transaction, commitment transaction family
DLC	Oracle signatures, contract settlement logic
Ordinals / Inscriptions	Transaction ordering and witness carriage
RGB	Bitcoin commitment points, client-side validation
Weakwork Claim	Overlay message, deterministic claim predicate, optional anchors
TXXM	Action, routing, validation, payload, references
Knot	Finite family of overlay events or claims
Braid	Ordered or partially ordered family of knots or claims
Accumulator	Compressed digest of overlay history subset
ACT	Accumulator roots and chain progress binding
AggWit Accounting Surface	Transaction or block data with extended weight/delta attribution
Merge-Mined Overlay Header	Base chain progress and auxiliary header logic
Provider Nodule	Relay role, policy role, market role, indexing role
Guild / Room / Submission	Overlay governance and discourse state objects

9 Multi-Diagram TikZ UML Package Views

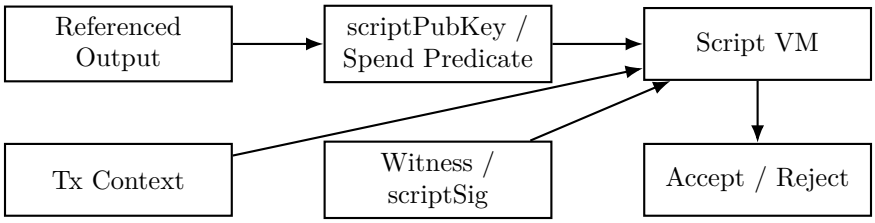
9.1 Diagram 1: Base Stack



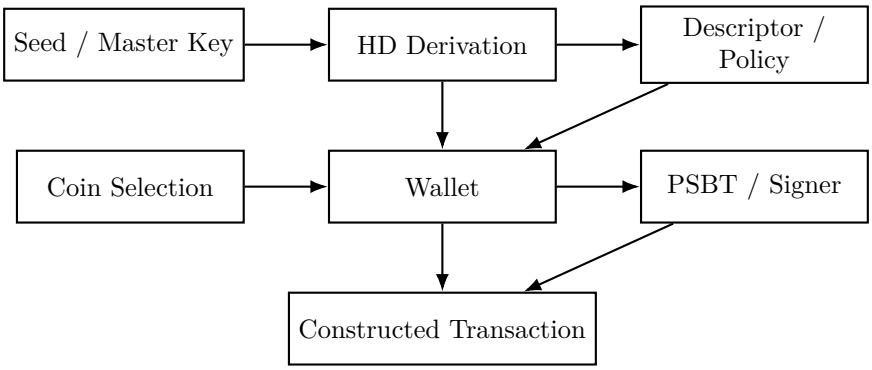
9.2 Diagram 2: Consensus-Native Core



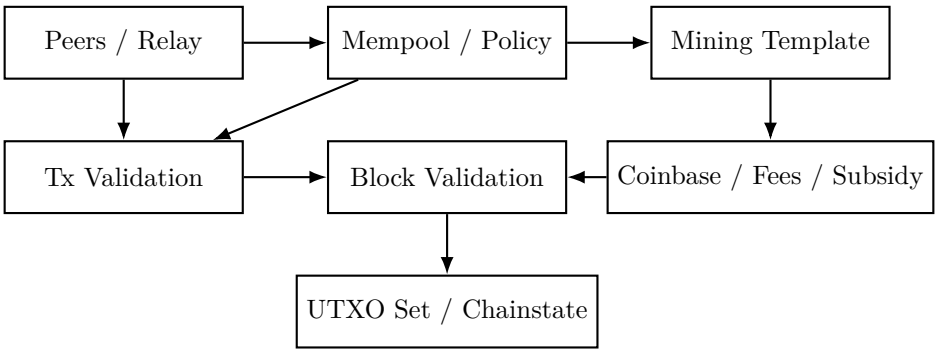
9.3 Diagram 3: Spend Authorization Surface



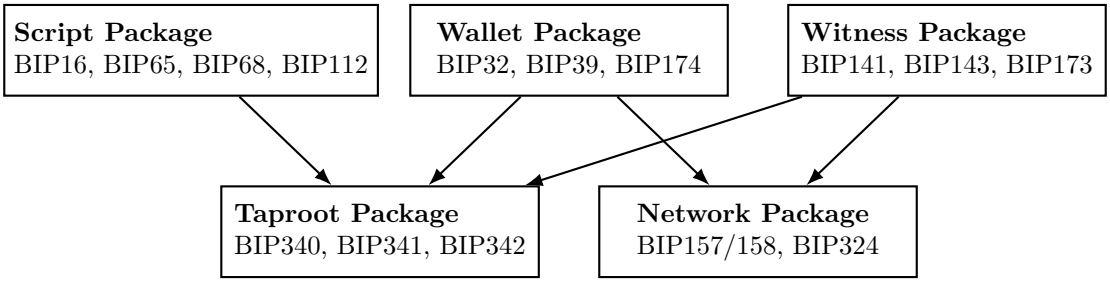
9.4 Diagram 4: Wallet and Policy Surface



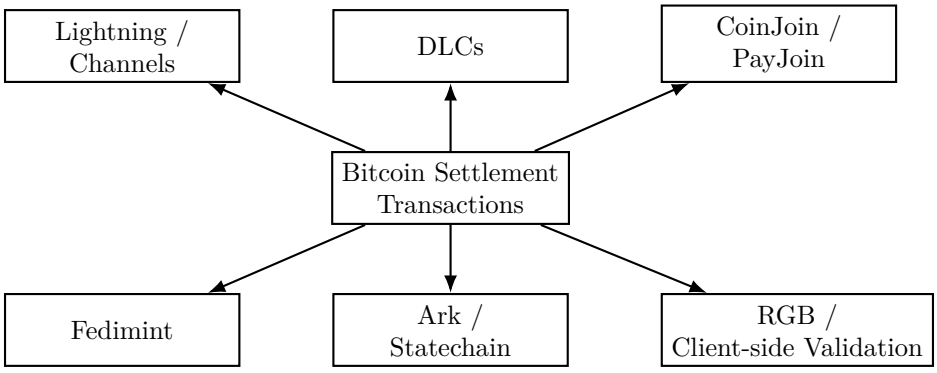
9.5 Diagram 5: Validation, Relay, and Mining



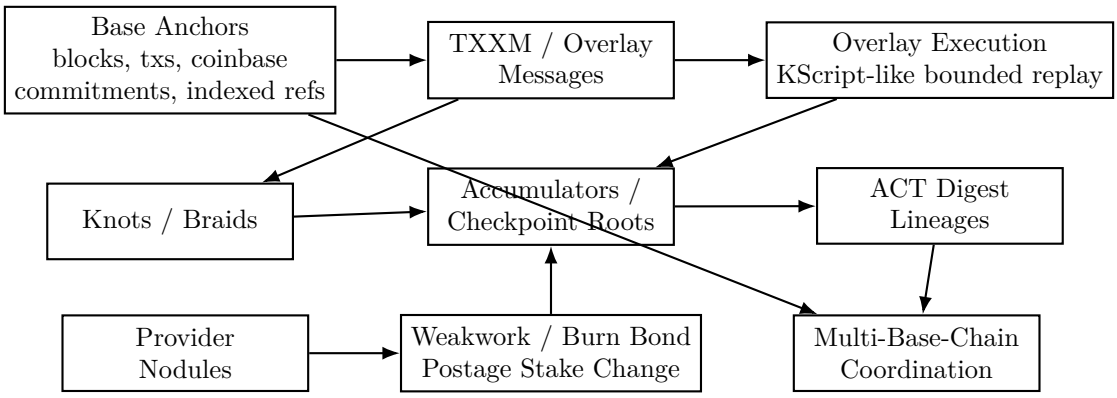
9.6 Diagram 6: Canonical BIP Package View



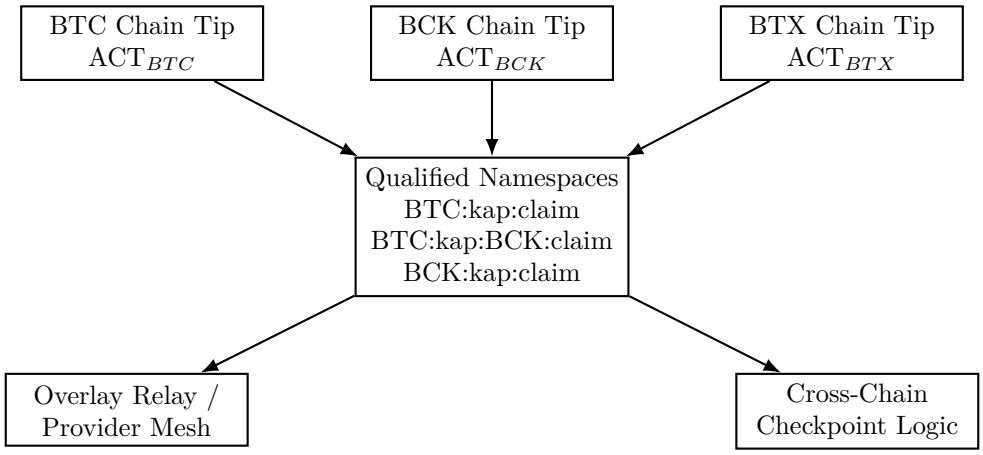
9.7 Diagram 7: Higher Protocols Above Settlement



9.8 Diagram 8: Kapnet Overlay Package View



9.9 Diagram 9: Multi-Chain Qualified Overlay Namespace



10 Main Structural Theorems

Theorem 4 (Bitcoin as a Layered Validity Machine)**.** *Bitcoin admits a decomposition into strata where lower layers define admissibility and upper layers define description, policy, transport, deployment, and optional higher-order coordination.*

Proof. The decomposition follows from the fact that cryptography precedes headers, headers precede blocks, outputs precede spend predicates, predicates precede wallets and validation policy, validated objects are transported over networking, mining assembles validated transactions, deployment modifies admissibility regimes, and higher protocols consume these surfaces without replacing them. □

Theorem 5 (Higher-Protocol Non-Equivalence)**.** *A higher Bitcoin protocol need not alter consensus semantics to be economically or coordination-relevant.*

Proof. Lightning, DLCs, proof systems, and anchoring schemes derive their effect from compositions of already-valid transactions, signatures, timelocks, or commitments. Their semantics are real for

participants without needing universal base-layer semantic awareness. □

Theorem 6 (Kapnet Compatibility)**.** *A Kapnet-style overlay is compatible with Bitcoin provided:*

1. *every overlay anchor is base-valid or deterministically derived therefrom,*
2. *replay is bounded and deterministic,*
3. *checkpoint compression preserves verifiability, and*
4. *overlay economics do not require unauthorized mutation of base consensus.*

Proof. Under these conditions, overlay state remains subordinate to base validity. Nodes can independently verify overlay histories against admissible anchors and accepted checkpoint rules without changing what counts as a valid Bitcoin block or transaction. □

Corollary 2 (Meta-Consensus Without Base Replacement)**.** *An overlay can implement rich routing, governance, messaging, checkpointing, and market logic without becoming the consensus substrate itself.*

11 AggWit and Extended Accounting as an Optional Surface

A Kapnet-aware map benefits from a place for AggWit-style accounting even when not assumed to be consensus-active on Bitcoin itself.

Definition 15 (Extended Weight Attribution)**.** *An extended weight attribution rule assigns internal cost, entropy, sanitation burden, or reward share to transactions or outputs based on richer accounting than raw base block weight.*

Remark 1. *Such accounting may be used:*

- *as pure overlay pricing,*
- *as node-local policy,*
- *as miner-side market logic,*
- *or, in a different chain family, as part of consensus-validity.*

Proposition 3 (Overlay Accounting Separation)**.** *Extended accounting can affect overlay fees and internal lotteries without altering base-validity unless explicitly embedded in consensus.*

12 Interpretive Summary

The entire map can be compressed into three statements.

1. Bitcoin base layers furnish scarce blockspace, globally replayable settlement, and deterministic admissibility.

2. Bitcoin-adjacent protocols exploit those guarantees for channels, contracts, privacy protocols, indexing systems, and application commitments.

3. A Kapnet-style overlay adds a further semantic and cypher-economic fabric: TXXMs, weakwork, knots, braids, ACTs, checkpoints, and multi-base-chain routing, all subordinate to anchored replay rules.

13 Conclusion

This document has done both requested upgrades. First, it restated the layered UML in a denser theorem/axiom style. Second, it expanded the single figure into multiple TikZ package and dependency diagrams. The resulting model is useful because it shows exactly where a concept lives.

If something changes what counts as a valid block or valid spend, it belongs low in the stack. If it describes, coordinates, routes, compresses, checkpoints, or prices behavior above the chain, it belongs high in the stack. Kapnet, in this framing, is neither hand-waved middleware nor a replacement blockchain. It is a structured overlay discipline for deterministic, replayable, economically legible coordination above Bitcoin-grade anchoring.